

Anonymous Method in C# with Examples

In this article, I am going to discuss the **Anonymous Method in C#** with Examples. Please read our previous article where we discussed **Generic Delegates in C#** with Examples. As part of this article, we are going to discuss the following pointers.

1. **What is Anonymous Method in C#?**
2. **Why do we need Anonymous Methods in C#?**
3. **Examples to understand Anonymous Method.**
4. **What are the Advantages of Using Anonymous Methods in C#?**
5. **Examples of Anonymous Methods Accessing Variables Defined in an Outer Function.**
6. **What are the Limitations of Anonymous Methods in C#?**

Note: Anonymous Methods are related to Delegates only. If you have not read our delegate articles, please read them before continuing with this article, otherwise, it will be difficult for you to understand the concept of Anonymous Methods.

What is Anonymous Method in C#?

As the name suggests, an anonymous method in C# is a method without a name. Or you can say a code block without having a name. **The Anonymous Methods in C# are defined using the delegate keyword and can be assigned to a variable of the delegate type.** If this is not clear at this moment, don't worry we will understand this with multiple examples.

Why do we need Anonymous Methods in C#?

In our **Delegates in C#** article, we discussed how to bind a delegate with a method. To bind a delegate with a method, first, we need to create an instance of the delegate and when we create the instance of the delegate, we need to pass the method name as a parameter to the delegate constructor, and it is the function the delegate will point to and when we invoke the delegate, this function which is pointed by the delegate is going to be executed. It is also possible that the delegates might be pointed to multiple functions and in that case, when we invoke the delegate instance, then all the functions which are pointed by the delegate are going to be executed.

Example to Understand Delegate in C#:

Before understanding the anonymous method, let us first see how we can use a delegate to execute the method. Why because to understand the Anonymous method, it is important to understand delegate because anonymous methods are used delegate. The following example shows how to declare a delegate, how to create an instance of a delegate, and how to invoke the delegate.

```
using System;
namespace DelegateDemo
{
    public class AnonymousMethods
    {
        public delegate string GreetingsDelegate(string name);
        public static string Greetings(string name)
        {
```

```

        return "Hello @" + name + " Welcome to Dotnet Tutorials";
    }
    static void Main(string[] args)
    {
        GreetingsDelegate gd = new GreetingsDelegate(AnonymousMethods.Greet);
        string GreetingsMessage = gd.Invoke("Pranaya");
        Console.WriteLine(GreetingsMessage);
        Console.ReadKey();
    }
}

```

When you run the above code, you will get the following output.

```

Hello @Pranaya Welcome to Dotnet Tutorials

```

In the above example,

1. We create one delegate (GreetingsDelegate)
2. Then we instantiate the delegate, while we are instantiating the delegate, we are passing the Method name (**Greetings**) as a parameter to the constructor of the delegate
3. Finally, we invoke the delegate

As of now, this is the approach we are following to bind a method to a custom delegate and execute it. An anonymous method in C# is also related to a delegate. Without binding a named block (function or method) to a delegate, we can also bind a code block (unnamed code block) to a delegate which is nothing but an anonymous method in C#.

Example to Understand Anonymous Methods in C#

The following example is the same as the previous example except here instead of binding the delegate with a named block (method or function), we are binding the delegate with an anonymous method (you can also unnamed code block).

```

using System;
namespace DelegateDemo
{
    public class AnonymousMethods
    {
        public delegate string GreetingsDelegate(string name);
        static void Main(string[] args)
        {
            GreetingsDelegate gd = delegate (string name)
            {
                return "Hello @" + name + " Welcome to Dotnet Tutorials";
            };
            string GreetingsMessage = gd.Invoke("Pranaya");
            Console.WriteLine(GreetingsMessage);
            Console.ReadKey();
        }
    }
}

```

```

    }
}
}

```

When you run the above code, you will get the same output as the previous example as shown in the below image.

```

Hello @Pranaya Welcome to Dotnet Tutorials

```

In the above example, the following code is an example of an Anonymous method. In this case, as you can see, instead of a method, we are binding the delegate to an unnamed code block which is also called an anonymous method and in C#, the anonymous methods are created by using the delegate keyword and if the anonymous method requires any input parameter, then you can pass the parameter within the parenthesis. Here, you can see, we are passing the string name parameter.

```

GreetingsDelegate gd = delegate (string name)
{
    return "Hello @" + name + " Welcome to Dotnet Tutorials";
};

```

What are the Advantages of Using Anonymous Methods in C#?

Lesser typing words. Generally, anonymous methods are suggested when the code volume is very less and if it is one-time use only. Now, let us proceed and try to understand the anonymous method with different examples by taking different scenarios.

Anonymous Method Accessing Variables Defined Outside:

First of all, can we access the variable within an anonymous method that is defined outside the anonymous method? Yes, we can access it. Please have a look at the following example for a better understanding. In the below example, you can see, we are accessing the Message variable within the anonymous method which is declared outside the anonymous method.

```

using System;
namespace DelegateDemo
{
    public class AnonymousMethods
    {
        public delegate string GreetingsDelegate(string name);
        static void Main(string[] args)
        {
            string Message = "Welcome to Dotnet Tutorials";
            GreetingsDelegate gd = delegate (string name)
            {
                return "Hello @" + name + " " + Message;
            }
        }
    }
}

```

```

    };
    string GreetingsMessage = gd.Invoke("Pranaya");
    Console.WriteLine(GreetingsMessage);
    Console.ReadKey();
}
}
}

```

Output: **Hello @Pranaya Welcome to Dotnet Tutorials**

What are the Limitations of Anonymous Methods in C#?

An anonymous method in C# cannot contain any jump statement like goto, break or continue.

```

GreetingsDelegate gd = delegate (string name)
{
    break;
    return message;
};

```

No enclosing loop out of which to break or continue

Anonymous Method in C# cannot access the ref or out parameter of an outer method.

```

public void SomeMethod(out int SomeVariable)
{
    SomeVariable = 10;
    string Message = "Welcome to Dotnet Tutorials";

    GreetingsDelegate gd = delegate (string name)
    {
        return "Hello @" + SomeVariable + " " + Message;
    };
}

```

Points to Remember while working with the Anonymous Methods in C#:

1. The anonymous methods are defined using the delegate keyword
2. An anonymous method must be assigned to a delegate type.

3. This method can access outer variables or functions except for the outer function ref and out parameter.
4. An anonymous method can be passed as a parameter.
5. This method can be used as an event handler. We have already discussed this in our [Events in C#](#) article.

Anonymous Method Real-Time Example in C#

As the name suggests, an anonymous method is a method without having a name i.e. it is an unnamed code block. Anonymous methods in C# are defined by using the delegate keyword and can be assigned to a variable of the delegate type that we have already discussed. In simple words, we can say that an anonymous method is a method without a name. Let us understand how a method can exist without a name in C# with one real-time example.

Step1:

Create a class file with the name Employee.cs and then copy and paste the following code into it. As you can see, it is a simple class having ID, Name, Gender, and Salary Properties.

```
namespace AnonymousMethodRealTimeExample
{
    // Step1
    // Create a class called Employee with
    // ID, Name, Gender and Salary Properties
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public double Salary { get; set; }
    }
}
```

Step2:

In our [Generic Delegates in C#](#) article, we have discussed that Predicate is a generic delegate that accepts a single input parameter of any data type and returns a Boolean value that is mandatory and fixed. That means the Predicate Generic Delegate in C# is used whenever your delegate returns a Boolean value, by taking only one input parameter. The following is the signature of the Predicate Generic Delegate.

```

namespace System
{
    //
    // Summary:
    //     Represents the method that defines a set of criteria and determines whether the
    //     specified object meets those criteria.
    //
    // Parameters:
    //     obj:
    //         The object to compare against the criteria defined within the method represented
    //         by this delegate.
    //
    // Type parameters:
    //     T:
    //         The type of the object to compare.
    //
    // Returns:
    //     true if obj meets the criteria defined within the method represented by this
    //     delegate; otherwise, false.
    public delegate bool Predicate<in T>(T obj);
}

```

As you can see in the above image, the Predicate Generic delegate takes one input parameter of type T and returns a Boolean value. Now, we need to use this Predicate Generic Delegate and hence we need to create a method whose signature must match the signature of the Predicate<Employee> delegate. Here, T is going to be our Employee class.

```

// Step 2:
// Create a method whose signature matches with the
// signature of Predicate<Employee> generic delegate
public static bool IsEmployeeExist(Employee Emp)
{
    return Emp.ID == 103;
}

```

As you can see in the above image, the IsEmployeeExist method takes one input parameter of the type Employee and returns a Boolean value. So, the above method signature matches the signature of the Predicate<Employee> Generic Delegate. This IsEmployeeExist method logic is very simple. It checks the ID value of the employee which is passed as a parameter to this function, if the ID value is 103, then it returns true else it returns false.

Step3:

Now, in this step, we are going to create an instance of the Predicate Generic Delegate. While we are creating the instance, we need to pass the IsEmployeeExist method as a parameter to the constructor of Predicate as shown in the image below.

```
// Step 3:  
// Create an instance of Predicate<Employee> delegate and  
// pass the method name as an argument to the delegate constructor  
Predicate<Employee> employeePredicate = new Predicate<Employee>(IsEmployeeExist);
```

Step4:

Now we need to create a list collection of type Employee to hold a list of Employees as shown in the below image. In our coming articles, we will discuss more on collections in C#.

```
//Step4  
//Create a collection of List of Employees  
List<Employee> listEmployees = new List<Employee>()  
{  
    new Employee{ ID = 101, Name = "Pranaya", Gender = "Male", Salary = 100000},  
    new Employee{ ID = 102, Name = "Priyanka", Gender = "Female", Salary = 200000},  
    new Employee{ ID = 103, Name = "Anurag", Gender = "Male", Salary = 300000},  
    new Employee{ ID = 104, Name = "Preety", Gender = "Female", Salary = 400000},  
    new Employee{ ID = 104, Name = "Sambit", Gender = "Male", Salary = 500000},  
};
```

Step5:

In this step, we need to pass the delegate instance i.e. employeePredicate to the Find method of the List collection class as shown in the image below.

```
// Step 5:  
// Now pass the delegate instance as the  
// argument to the Find() method of List collection  
Employee employee = listEmployees.Find(x => employeePredicate(x));  
Console.WriteLine(@"ID : {0}, Name : {1}, Gender : {2}, Salary : {3}",  
    employee.ID, employee.Name, employee.Gender, employee.Salary);
```

In this case, for each employee object, the delegate instance employeePredicate is going to be invoked and internally it will call the IsEmployeeExist method by passing the employee object and if that employee object exists it will return true else it will return false. As we have written the logic, the employee whose id is 103, for that employee the IsEmployeeExist method returns true and that employee information is going to be stored inside the left-hand side employee variable and then we are just displaying the ID, Name, Gender, and Salary information of that employee.

Now, we have already created the Employee class inside the Employee.cs class file. Now, modify the Program.cs class file as follows to implement the above-discussed example.

```
using System;  
using System.Collections.Generic;
```

```

namespace AnonymousMethodRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            // Step 3:
            // Create an instance of Predicate<Employee> delegate and
            // pass the method name as an argument to the delegate constructor
            Predicate<Employee> employeePredicate = new Predicate<Employee>
(IsEmployeeExist);
            //Step4
            //Create a collection of List of Employees
            List<Employee> listEmployees = new List<Employee>()
            {
                new Employee{ ID = 101, Name = "Pranaya", Gender = "Male", Salary = 100000},
                new Employee{ ID = 102, Name = "Priyanka", Gender = "Female", Salary = 200000},
                new Employee{ ID = 103, Name = "Anurag", Gender = "Male", Salary = 300000},
                new Employee{ ID = 104, Name = "Preety", Gender = "Female", Salary = 400000},
                new Employee{ ID = 104, Name = "Sambit", Gender = "Male", Salary = 500000},
            };
            // Step 5:
            // Now pass the delegate instance as the
            // argument to the Find() method of List collection
            Employee employee = listEmployees.Find(x => employeePredicate(x));
            Console.WriteLine(@"ID : {0}, Name : {1}, Gender : {2}, Salary : {3}",
                employee.ID, employee.Name, employee.Gender, employee.Salary);
            Console.ReadKey();
        }
        // Step 2:
        // Create a method whose signature matches with the
        // signature of Predicate<Employee> generic delegate
        public static bool IsEmployeeExist(Employee Emp)
        {
            return Emp.ID == 103;
        }
    }
}

```

Output: ID : 103, Name : Anurag, Gender : Male, Salary : 300000

Using the Anonymous Method in C#

As of now, we did the following.

1. We create a method whose signature matches with the Predicate Generic Delegate
2. Then we create an instance of the Predicate Generic Delegate
3. Then we pass that Predicate Generic Delegate Instance as an argument to the Find method of the List collection class

Using the anonymous method, we can safely avoid the above three steps. We can pass an anonymous method as an argument to the Find() method as shown in the below code.

```
Employee employee = listEmployees.Find(
    delegate (Employee x)
    {
        return x.ID == 103;
    }
);
```

 Anonymous Method passing to the Find method

With the above anonymous method, you can simply remove the step and step2 and you can replace step5 with the above code. So, with the above changes in place, now the Program class code should look as follows.

```
using System;
using System.Collections.Generic;
namespace AnonymousMethodRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step2
            //Create a collection of List of Employees
            List<Employee> listEmployees = new List<Employee>()
            {
                new Employee{ ID = 101, Name = "Pranaya", Gender = "Male", Salary = 100000},
                new Employee{ ID = 102, Name = "Priyanka", Gender = "Female", Salary = 200000},
                new Employee{ ID = 103, Name = "Anurag", Gender = "Male", Salary = 300000},
                new Employee{ ID = 104, Name = "Preety", Gender = "Female", Salary = 400000},
                new Employee{ ID = 104, Name = "Sambit", Gender = "Male", Salary = 500000},
            };
            // Step3
```

```

// An anonymous method is being passed as an argument to
// the Find() method of List Collection.
Employee employee = listEmployees.Find(
    delegate (Employee x)
    {
        return x.ID == 103;
    }
);
Console.WriteLine(@"ID : {0}, Name : {1}, Gender : {2}, Salary : {3}",
    employee.ID, employee.Name, employee.Gender, employee.Salary);
Console.ReadKey();
    }
}
}

```

Output: ID : 103, Name : Anurag, Gender : Male, Salary : 300000

Find Method in C#:

In the above two examples, the **Find()** method of the generic List collection class expects a delegate to be passed as an argument. If you want to look at the signature of the Find method, then right-click on the **Find()** method and select **Go To Definition** from the context menu. Then you will see the following method signature of the Find method. You can see it is taking one parameter of generic Predicate.

```

// Summary:
//   Searches for an element that matches the conditions defined by the specified
//   predicate, and returns the first occurrence within the entire System.Collections.Generic.List`1.
//
// Parameters:
//   match:
//   The System.Predicate`1 delegate that defines the conditions of the element to
//   search for.
//
// Returns:
//   The first element that matches the conditions defined by the specified predicate,
//   if found; otherwise, the default value for type T.
//
// Exceptions:
//   T:System.ArgumentNullException:
//   match is null.
public T Find(Predicate<T> match);

```

Note: Once we start discussing [Generic Collection Classes in C#](#), then you will get better clarity about the Find method.

In the next article, I am going to discuss the [Lambda Expression in C#](#) with Examples. In this article, I try to explain the **Anonymous Method in C# with Examples**. I hope you understood the need and use of Anonymous Methods in C# with Examples.